社内スタート計画 — シャチョツーで自社を回す

> **シャチョツーMVP完成前から、CLIベースで11エージェントを即座に稼働させる。**
> MVP完成後にシャチョツーに移行し、そのままドッグフーディング + 最強の営業デモにする。
>
> **Phase Aの11エージェントは暫定構成。シャチョツーの動的エージェントモデル（BPO編）が正の設計であり、Phase B以降はそちらに合流する。Phase Aの経験は「ゲノムの推奨初期エージェントセット」としてプロダクトに還元する。**

---

1. 戦略概要

実行順序

```
Step 1: 現職（コンサル・人材紹介会社）で Phase A を実行
        → 11エージェントを CLI で構築・運用
        → 「コンサル・人材紹介業ゲノム」が1つ目のゲノムになる

Step 2: 現職を「1社目の顧客モデル」として実績化
        → シャチョツー MVP に Phase B 移行
        → 「うちで回してます」が営業トーク

Step 3: スピンアウト → 独立
        → 新会社（シャチョツー社）自体もシャチョツーで回す
        → 「SaaS業ゲノム」が2つ目のゲノム
        → 2業種のゲノムが揃った状態でローンチ
```

なぜやるか

1. **ドッグフーディング**: 自社で使い倒すことでプロダクトの品質を担保
2. **最強のデモ**: 「自社で全部回してます」が営業トークになる
3. **限界突破**: 社長・戦略・営業・マーケ・採用・経理を1人で回すのは不可能。AIで回す
4. **MVP開発中も価値が出る**: Phase A は CLI で即日稼働。MVP完成を待たない
5. **ゲノムが2つ揃った状態で独立**: コンサル・人材紹介業 + SaaS業

なぜ Phase A（CLI）が必要か

Phase B（シャチョツーMVP上での稼働）にはプロダクトが必要。プロダクトはまだない。

	Phase A なし	Phase A あり
MVP開発中	エージェント支援ゼロ。全部手動	CLIエージェントが即日稼働
MVP設計品質	机上の空論リスク	実運用で検証済みの知見が設計に反映
現職での実績	なし	「AIエージェントで業務効率化」の実績

---

2. フェーズ構成

```
Phase A（CLI構築）  →  Phase A'（初期顧客共創）  →  Phase B（MVP移行）  →  Phase C（完全自律）
  現職で11エージェント      3社を同時並行で巻き込み      シャチョツーMVPに移行      スピンアウト後
  CLI + 既存SaaSで構築      建設・製造・歯科ゲノム作成    ゲノム入りでローンチ       自社完全移行
```

フェーズ	舞台	内容	前提条件
Phase A	現職（コンサル・人材紹介）	11エージェント構築（CLI + 既存SaaS）	なし。既存ツールで構築
Phase A'	Phase Aと同時並行	初期顧客3社（建設・製造・歯科）とゲノム共創	Phase A着手後すぐ開始
Phase B	現職 + 初期顧客	シャチョツーMVPに移行。ゲノム入りでローンチ	シャチョツーの Brain + Knowledge 稼働
Phase C	スピンアウト後の新会社	自社もシャチョツーで完全自律運用	Workers(BPO) + Proactive 稼働

ゲノム構築の順序

順序	ゲノム	構築元	フェーズ
1st	コンサル・人材紹介業	現職（Phase A）	Phase A
2nd	建設業	初期顧客（Phase A'）	Phase A'
3rd	製造業	初期顧客（Phase A'）	Phase A'
4th	歯科	初期顧客（Phase A'）	Phase A'
5th	SaaS業	スピンアウト後の自社（Phase C）	Phase C

---

3. Phase A — CLI + 既存ツールで即時構築

技術基盤

要素	選定	理由
LLM	Claude CLI (claude-code)	MCP禁止。CLI駆動で全操作
タスク管理	GitHub Issues + Projects	開発と統合。エージェントがIssue操作
ドキュメント	Git管理のMarkdown	Worktreeベースのため。外部依存を減らす
コミュニケーション	Slack CLI (slack-cli)	通知・レポート送信
経理	freee API	仕訳・請求書・経費
監視	GCP Cloud Monitoring	アラート → Slack通知

11エージェント構成

コンサル・人材紹介会社での優先度順:

優先度	エージェント	概要	理由
最重要	Sales Agent	リード管理・提案書生成・フォロー	案件獲得が生命線
最重要	Hiring Agent	人材スクリーニング・JD・マッチング	人材紹介の本業
高	CS Agent	クライアントフォロー・チャーン防止	既存顧客維持
高	Analysis Agent	市場分析・競合監視・KPIトラッキング	案件・候補者市場の把握
高	Strategy Agent	経営判断・優先順位付け・リスク評価	全体の意思決定
中	Marketing Agent	コンテンツ生成・SNS・SEO	リード獲得
中	Legal Agent	契約書レビュー・規約管理	人材紹介契約の管理
中	Accounting Agent	仕訳・請求書・経費・資金繰り	月次決算
低	Product Agent	フィードバック集約・ロードマップ	シャチョツー開発用
低	DevOps Agent	インフラ監視・リリース管理	シャチョツー開発用
低	PR Agent	プレスリリース・メディア	スピンアウト後に本格化

> **これら11エージェントの知識 = シャチョツーの「コンサル・人材紹介業ゲノム」そのもの。**
> Phase Bでは genome_templates テーブルに格納される。

各エージェント仕様

1. Sales Agent（営業）— 最重要

```
入力: リード情報、商談メモ、業界データ、顧客の課題ヒアリングシート
処理: Claude CLI で提案書生成、フォローメール下書き、パイプライン更新
出力: .context/sales/ に提案書・パイプライン・顧客分析
頻度: 毎日（パイプライン更新）+ 商談発生時（提案書生成）
```

Phase Aでやること:
- 提案書テンプレート（業種別: コンサル案件・人材紹介案件）
- パイプライン管理（.context/sales/pipeline.md）
- フォローメール自動下書き
- 商談議事録 → 次アクション自動抽出

2. Hiring Agent（採用・人材）— 最重要

```
入力: 求人票、候補者レジュメ、スキルシート、面談フィードバック
処理: Claude CLI でJD生成、候補者スクリーニング、マッチングスコアリング
出力: .context/hiring/ にJD・候補者評価・マッチング結果
頻度: 案件受注時（JD生成）+ 候補者応募時（スクリーニング）
```

Phase Aでやること:
- JD自動生成（職種・レベル・業界別テンプレート）
- 候補者レジュメの構造化 + スキルマッチング
- 面談後のフィードバック集約・次ステップ提案
- クライアント×候補者のマッチングレポート

3. CS Agent（カスタマーサクセス / サポート）

```
入力: クライアントからの問い合わせ、サービス利用状況、NPS
処理: Claude CLI でFAQ検索、回答ドラフト生成、チャーンリスク分析
出力: .context/cs/ に回答ドラフト・チャーンアラート
頻度: 問い合わせ発生時（即時）+ 週次チャーンレビュー
```

Phase Aでやること:
- FAQ自動生成
- 問い合わせ分類・優先度付け
- オンボーディングチェックリスト自動生成

4. Analysis Agent（戦略・分析）

```
入力: 求人市場データ、競合動向、社内KPI
処理: Claude CLI で市場レポート生成、競合分析、SWOT更新
出力: .context/analysis/ に競合レポート・市場トレンド
頻度: 週次 + 競合動向検知時
```

Phase Aでやること:
- 競合ウォッチリスト管理
- 市場トレンドの定期クロール + 要約
- SWOT分析の自動更新

5. Strategy Agent（経営・意思決定）

```
入力: 全エージェントの週次レポート、KPIデータ、市場情報
処理: Claude CLI で週次ブリーフィング生成、優先順位提案、リスク評価
出力: .context/strategy/ に週次レポート・優先順位・リスクアラート
頻度: 日次ブリーフィング + 週次KPIレビュー
```

Phase Aでやること:
- 週次経営レポートの自動生成
- KPI異常検知 → アラート
- 優先順位の定期見直し提案

6. Marketing Agent（マーケティング）

```
入力: ターゲット業界情報、コンテンツカレンダー、SEOデータ
処理: Claude CLI でブログ記事・SNS投稿・メルマガ生成
出力: .context/marketing/ にコンテンツドラフト
頻度: 週次（コンテンツ生成）+ 月次（カレンダー更新）
```

7. Legal Agent（法務・契約）

```
入力: 契約書テンプレート、人材紹介契約、NDA、業務委託契約
処理: Claude CLI で契約書レビュー、リスク条項チェック
出力: .context/legal/ にレビュー結果・契約ドラフト
頻度: 契約発生時 + 四半期の規約レビュー
```

8. Accounting Agent（経理）

```
入力: freee APIからの取引データ、請求書、経費レシート
処理: Claude CLI + freee API で仕訳提案生成、キャッシュフロー分析
出力: .context/accounting/ に仕訳ドラフト・月次PL
頻度: 毎日（仕訳）+ 月次（PL・キャッシュフロー）
```

9. Product Agent（プロダクトマネジメント）

```
入力: ユーザーフィードバック、GitHub Issues、競合機能比較
処理: Claude CLI でフィードバック分類、ロードマップ優先順位提案
出力: .context/product/ にフィードバック分析・ロードマップ更新案
頻度: 週次
```

10. DevOps Agent（開発運用）

```
入力: GCP監視アラート、GitHub Actions結果、依存脆弱性情報
処理: Claude CLI でインシデント分析、リリースノート生成
出力: .context/devops/ にインシデントレポート・リリースノート
頻度: アラート発生時（即時）+ リリース時 + 週次セキュリティスキャン
```

11. PR Agent（広報）

```
入力: プロダクトリリース情報、業界ニュース、メディアリスト
処理: Claude CLI でプレスリリースドラフト、メディアピッチ作成
出力: .context/pr/ にプレスリリース・ピッチメール下書き
頻度: リリース時 + 月次メディアアプローチ
```

.context/ ディレクトリ構成

```
.context/
├── strategy/       # Strategy Agentの出力
├── sales/          # Sales Agentの出力
├── marketing/      # Marketing Agentの出力
├── hiring/         # Hiring Agentの出力
├── accounting/     # Accounting Agentの出力
├── analysis/       # Analysis Agentの出力
├── cs/             # CS Agentの出力
├── legal/          # Legal Agentの出力
├── product/        # Product Agentの出力
├── devops/         # DevOps Agentの出力
├── pr/             # PR Agentの出力
├── events/         # エージェント間イベント（JSONL）
├── logs/           # 監査ログ
├── escalation/     # エスカレーション記録
└── tmp/            # 一時作業用（タスク完了時に掃除）
```

スクリプト仕様

Phase Aでは `scripts/` 配下に11本のシェルスクリプトを作る。

共通仕様

```
#!/bin/bash
set -euo pipefail

# 共通: コスト上限チェック
check_cost_budget() {
    TODAY_COST=$(cat .context/logs/daily_cost.json | jq '.total_usd')
    if (( $(echo "$TODAY_COST > 15" | bc -l) )); then
        echo "ERROR: Daily cost limit exceeded ($TODAY_COST). Escalating."
        slack_notify "コスト上限超過: $TODAY_COST USD"
        exit 1
    fi
}

# 共通: 監査ログ記録
log_action() {
    local agent=$1 action=$2 output_path=$3
    jq -n \
        --arg ts "$(date -u +%Y-%m-%dT%H:%M:%SZ)" \
        --arg agent "$agent" \
        --arg action "$action" \
        --arg output "$output_path" \
        '{timestamp:$ts, agent:$agent, action:$action, output_path:$output}' \
        >> .context/logs/audit.jsonl
}
```

スクリプト一覧・実行スケジュール

スクリプト	用途	承認要否	cron設定
`strategy_report.sh`	週次経営レポート	必要	`0 7 * * 1`
`sales_proposal.sh`	提案書生成	不要（送付は人間）	イベント駆動
`marketing_content.sh`	コンテンツ生成	必要（公開前）	イベント駆動
`hiring_jd.sh`	JD生成・候補者スクリーニング	必要（公開前）	イベント駆動
`accounting_sync.sh`	freee取引同期・仕訳	必要	`30 17 * * *`
`analysis_competitor.sh`	競合モニタリング	不要	`0 6 * * 3`
`cs_support.sh`	問い合わせ回答ドラフト	必要（送信前）	イベント駆動
`legal_review.sh`	契約書リスクチェック	常に必要	イベント駆動
`product_feedback.sh`	フィードバック集約	不要	`0 15 * * 4`
`devops_monitor.sh`	インフラ・セキュリティ	アラート時	`30 6 * * *`
`pr_release.sh`	プレスリリース生成	必要（公開前）	イベント駆動

初期化スクリプト

```
# scripts/init_context.sh — Phase A開始時に一度だけ実行
#!/bin/bash
DIRS=(strategy sales marketing hiring accounting analysis cs legal product devops pr events logs escalation tmp)
for dir in "${DIRS[@]}"; do
    mkdir -p ".context/$dir"
    touch ".context/$dir/.gitkeep"
done
echo '{"total_usd": 0, "date": "'$(date +%Y-%m-%d)'"}' > .context/logs/daily_cost.json
echo "initialized .context/"
```

Phase A 成功基準

- [ ] 全11エージェントが週次で稼働している
- [ ] 人間の介入が「承認のみ」になっている機能が3つ以上
- [ ] エージェントの判断ログがgitで追跡可能
- [ ] .context/ 経由のエージェント間連携が機能している
- [ ] Phase A' の初期顧客3社の受け入れ準備が完了

---

4. Phase B — シャチョツーMVP移行

移行マッピング

Phase Aの構成	Phase Bの移行先	移行難易度
Sales Agent (CLI)	Brain → Knowledge + Genome	低
Hiring Agent (CLI)	Brain → Knowledge + Genome	低
CS Agent (CLI)	Brain → Knowledge + Proactive	中
Analysis Agent (CLI)	Brain → Proactive + Twin	中
Strategy Agent (CLI)	Brain → Twin + Proactive	中
Marketing Agent (CLI)	Brain → Knowledge + Extraction	低
Legal Agent (CLI)	Brain → Knowledge + Extraction	低
Accounting Agent (freee API)	Workers → BPO + Connector	高
Product Agent (CLI)	Brain → Knowledge + Proactive	低
DevOps Agent (CLI)	Workers → BPO + Connector	高
PR Agent (CLI)	Brain → Knowledge + Extraction	低

移行手順（各エージェント共通）

```
Step 1: データ移行
  .context/{agent}/ のMarkdownデータ
    → brain/ingestion/ でシャチョツーに取り込み
    → brain/extraction/ で構造化
    → knowledge_items テーブルに格納

Step 2: 機能移行
  scripts/{agent}.sh のCLI処理
    → routers/ の対応APIに置き換え
    → フロントエンドUIから操作可能に

Step 3: 検証
  Phase Aの出力品質 vs Phase Bの出力品質を比較
    → 品質が同等以上なら移行完了
    → 劣化があればフィードバックしてチューニング
```

Phase B 成功基準

- [ ] 11エージェント中7つ以上がシャチョツー上で稼働
- [ ] Phase Aと同等以上の出力品質
- [ ] 外部顧客にデモ可能な状態
- [ ] 「自社で使っている」と言える実績データが蓄積
- [ ] ゲノムが2業種以上シャチョツー上で稼働

---

5. Phase C — 完全自律運用

自律レベル定義

```
Level 0: 人間が指示 → AIが実行
Level 1: AIが提案 → 人間が承認 → AIが実行
Level 2: AIが実行 → 人間に事後報告（異常時のみ介入）
Level 3: AIが実行 → ログ記録のみ（人間は監査時に確認）
```

エージェント別の目標自律レベル

エージェント	Phase A	Phase B	Phase C目標
Strategy Agent	Level 0	Level 1	Level 1（意思決定は常に人間承認）
Sales Agent	Level 0	Level 1	Level 2（提案書生成は自動、送信は承認）
Marketing Agent	Level 0	Level 1	Level 2（コンテンツ生成は自動、公開は承認）
Hiring Agent	Level 0	Level 1	Level 2（スクリーニングは自動、面談判断は承認）
Accounting Agent	Level 0	Level 0	Level 2（仕訳は自動、支払い実行は承認）
Analysis Agent	Level 0	Level 1	Level 3（分析・レポートは完全自動）
CS Agent	Level 0	Level 1	Level 2（FAQ回答は自動、エスカレーションは承認）
Legal Agent	Level 0	Level 0	Level 1（契約は常に人間承認）
Product Agent	Level 0	Level 1	Level 2（分析は自動、ロードマップ変更は承認）
DevOps Agent	Level 0	Level 1	Level 2（監視・アラートは自動、本番操作は承認）
PR Agent	Level 0	Level 1	Level 2（ドラフトは自動、公開は承認）

絶対にLevel 3にしない操作:
- 資金移動・支払い実行
- 外部への契約・法的拘束力のある送信
- 採用の最終判断
- 経営方針の変更

Phase C 成功基準

- [ ] 社長の日次作業が「レビュー + 承認」のみ（30分以内）
- [ ] 全11エージェントの稼働率95%以上
- [ ] エージェント判断の精度90%以上（人間修正率10%以下）
- [ ] 外部顧客3社以上にライブデモ実施
- [ ] 「自社運用実績」としてケーススタディ公開

---

6. エージェント規約（全フェーズ共通）

> **このセクションは全AIエージェントの行動規範。**
> エージェント起動時に必ず参照すること。

6.1 MCP禁止

- **MCPは一切使用しない**
- 全操作はCLI経由で行う（Claude CLI, gh, slack-cli, freee-cli, gcloud 等）
- 理由: MCPは抽象度が高すぎて、操作の透明性・再現性・デバッグ性が落ちる

6.2 Memory管理

- **セッション間のMemoryは使用しない**（Worktreeベースの開発のため）
- 永続化が必要な情報は**全てgit管理**する
- 設計判断 → `shachotwo/adr/`
- エージェント出力 → `.context/{agent}/`
- 運用ログ → `.context/logs/`

6.3 一時ファイル

- **一時ファイルは `.context/tmp/` に作成する**
- `/tmp/` や OS のテンポラリディレクトリは**使用禁止**
- 理由: Subagentや別Agentが一時ファイルを読む必要がある

6.4 ADR（Architecture Decision Records）

- **大きめの設計変更・技術選定は必ずADRを作成**
- 保存先: `shachotwo/adr/`

6.5 Plan Review

- **Planを人間に提示する前に、AIセルフレビューを実施する**
- レビュー観点: 技術的実現可能性、既存設計との整合性、スコープの妥当性

6.6 Fail-safe原則

- **判断に迷ったら実行しない。人間にエスカレーションする**
- エスカレーション: Slack通知 → `.context/escalation/` に記録 → タスク一時停止

6.7 変更の原子性

- **1タスク = 1コミット**
- コミットメッセージは `feat:`, `fix:`, `chore:`, `docs:` プレフィックス + 日本語OK

6.8 出力の検証義務

- **LLM出力をそのまま外部に送信しない**
- 事実確認、トーン確認、PII漏洩チェック、社内情報の意図しない露出チェック

6.9 コスト上限

- 通常タスク: $1.00 / タスク
- 大規模分析: $5.00 / タスク（事前承認必要）
- 月間上限: $500
- 無限ループ防止: 同一プロンプトの再試行は最大3回まで

6.10 秘密情報の扱い

- **ローカル環境**: 環境変数（`.env`）で管理。`.env` はgitignoreに入れる
- **本番環境**: GCP Secret Manager のみ
- APIキー・トークンをソースコードに書くこと・シークレットをログに出力すること・`.env` をコミットすることは禁止

6.11 並列実行の排他制御

- `.context/locks/{resource_name}.lock` で排他制御
- ロック取得できない場合は待機（最大5分）→ タイムアウトでエスカレーション

6.12 ログの構造化

- 全エージェントの実行ログは**JSON Lines（`.jsonl`）**で `.context/logs/` に記録
- 「なぜこの判断をしたか」を後から追跡可能にする

6.13 ゲノム・ナレッジのフィードバック義務

- Phase Aで得た知見は、シャチョツーのゲノムテンプレートにフィードバックする
- フィードバック先: `.context/product/feedback/` に記録 → Product Agentが集約

---

7. 運用ルール

Human-in-the-Loop 基準

必ず人間承認が必要な操作（例外なし）：

カテゴリ	具体例
金銭	支払い実行、振込、請求書送付、契約締結
対外コミュニケーション	顧客へのメール送信、SNS投稿の公開
採用	面談の合否判断、オファー送付
経営判断	事業方針変更、予算配分
法的行為	契約書の署名、規約変更
インフラ	本番環境へのデプロイ、DBスキーマ変更

AIが自律実行してよい操作:

カテゴリ	具体例	条件
情報収集	市場調査、競合モニタリング	外部APIの利用範囲内
ドラフト生成	提案書、メール、記事、JD	送信・公開はしない
分析	KPI集計、トレンド分析	社内データのみ
整理	ファイル整理、タグ付け	削除はしない
通知	Slackへの社内通知	社内チャネルのみ

承認フロー

```
AIエージェント
  ├─ 自律実行可能 → 実行 → ログ記録
  └─ 承認必要 → .context/{agent}/pending/ にドラフト配置
                  → Slack通知「承認待ち: {概要}」
                  → 社長が確認
                      ├─ 承認 → 実行 → ログ記録
                      ├─ 修正指示 → AI修正 → 再承認フロー
                      └─ 却下 → ログ記録（却下理由）
```

監査ログフォーマット

```
{
  "timestamp": "2026-03-12T09:00:00+09:00",
  "agent": "sales_agent",
  "action": "generate_proposal",
  "input_summary": "顧客X向け提案書ドラフト生成",
  "output_path": ".context/sales/proposals/customer_x_v1.md",
  "decision_reasoning": "顧客Xの業種テンプレート適用",
  "approval_required": false,
  "llm_model": "claude-opus-4-6",
  "llm_tokens_used": 4500,
  "llm_cost_usd": 0.12,
  "duration_sec": 8.3
}
```

エージェント間イベント連携

```
{
  "event_id": "evt_20260312_001",
  "source_agent": "cs_agent",
  "event_type": "churn_risk_detected",
  "severity": "high",
  "target_agents": ["strategy_agent", "sales_agent"],
  "payload": {
    "company": "クライアントA",
    "risk_score": 8,
    "signal": "7日間未連絡 + 問い合わせ件数50%減"
  },
  "created_at": "2026-03-12T09:00:00Z",
  "processed_by": []
}
```

.context/ 読み書き契約

```
凡例:
  W: このエージェントが書く
  R: このエージェントが読む
  E: エスカレーション時のみ書く
```

ファイル / ディレクトリ	Strategy	Sales	Marketing	Hiring	Accounting	Analysis	CS	Legal	Product	DevOps	PR
`strategy/weekly_report.md`	W	R	R	R	R	R	-	-	R	-	R
`strategy/priorities.md`	W	R	R	R	-	R	R	-	R	R	R
`sales/pipeline.md`	R	W	-	-	R	R	-	-	-	-	-
`sales/proposals/`	-	W	-	-	-	-	-	R	-	-	-
`marketing/content_calendar.md`	R	R	W	-	-	-	-	-	-	-	R
`hiring/candidates.md`	R	-	-	W	-	-	-	-	-	-	-
`hiring/jd/`	-	-	R	W	-	-	-	-	-	-	-
`accounting/monthly_pl.md`	R	R	-	-	W	R	-	-	-	-	-
`accounting/cashflow.md`	R	-	-	-	W	R	-	-	-	-	-
`analysis/competitor_report.md`	R	R	R	-	-	W	-	-	R	-	R
`analysis/market_trends.md`	R	R	R	-	-	W	-	-	R	-	-
`cs/churn_alerts.md`	R	R	-	-	-	-	W	-	R	-	-
`cs/feedback_raw.md`	-	-	-	-	-	-	W	-	R	-	-
`legal/contracts/`	R	R	-	R	R	-	-	W	-	-	-
`product/roadmap.md`	R	R	R	-	-	-	-	-	W	R	-
`product/feedback/`	-	-	-	-	-	-	R	-	W	-	-
`devops/incident_log.md`	R	-	-	-	-	-	R	-	-	W	-
`devops/release_notes.md`	-	R	R	-	-	-	-	-	R	W	R
`pr/press_releases/`	R	-	-	-	-	-	-	-	R	-	W
`escalation/`	R	E	E	E	E	E	E	E	E	E	E
`logs/audit.jsonl`	R	R	R	R	R	R	R	R	R	R	R

---

8. 製品との関係

Phase Aの知識 → シャチョツーのゲノムになる

Phase Aの成果物	シャチョツーでの位置づけ
11エージェントの入出力仕様	`genome_templates` の推奨初期エージェントセット
.context/ の読み書き契約	エージェント間連携の設計パターン
スクリプトの処理ロジック	`workers/bpo/` の操作テンプレート
運用ルール・承認フロー	`brain/proactive/` の承認ルール
監査ログ形式	`audit_logs` テーブルのスキーマ

フィードバックループ

```
Phase A 運用（CLI）
    ↓
.context/product/feedback/ に記録
    ↓
Product Agent が週次で集約
    ↓
shachotwo/ の設計ドキュメント更新 or GitHub Issue 作成
    ↓
シャチョツー MVP に反映
    ↓
Phase B で移行 → さらに改善（ループ）
```

---

9. 懸念事項と対策

IP（知的財産）の帰属

現職で作ったコード・ゲノムのスピンアウト時の扱い:
- **コード**: 業務時間外に個人リポジトリで開発 or 現職と事前合意
- **ゲノム（パターン・テンプレート）**: 匿名化したパターンは持ち出し可能にする合意を取る
- **現職のデータ**: 顧客情報・候補者情報は持ち出し不可。ゲノム（匿名化済み）のみ

現職のデータ保護

- 顧客情報・候補者の個人情報は `.context/` に残さない
- ゲノムテンプレートには一般化・匿名化したパターンのみ格納
- `.env` に格納した認証情報は絶対にコミットしない

現職の理解

- 社内にAIエージェントを入れることへの事前合意が必要
- 「業務効率化の実験」として位置づける
- データの取り扱いについて透明性を確保する

---

10. 11エージェントの役割定義

> Phase Aで社内運用するCLIエージェント群。
> 各エージェントは独立したPythonスクリプト or Makefileターゲットとして実装。
> orchestratorが全体を調整する。

エージェント一覧

#	エージェント名	担当領域	実行頻度	依存先
1	knowledge-collector	社長との対話からナレッジを構造化	随時（対話発生時）	llm/, brain/extraction/
2	qa-engine	社内Q&A応答	随時（質問発生時）	brain/knowledge/, llm/
3	process-miner	SaaS操作ログから業務フローを発見	日次バッチ	workers/connector/, brain/inference/
4	bpo-executor	定型業務の自動実行（Shadow→承認付き）	イベント駆動	workers/bpo/, process-miner
5	report-generator	日次/週次レポート生成	日次 8:00 / 週次 月曜 9:00	brain/knowledge/, brain/twin/
6	risk-detector	リスク検出・アラート	6時間ごと	brain/proactive/, brain/twin/
7	genome-builder	業種テンプレート構築・更新	週次	brain/genome/, brain/knowledge/
8	saas-connector	SaaS API接続管理・ヘルスチェック	15分ごと	workers/connector/
9	audit-logger	全操作の監査ログ記録	常時（イベント駆動）	security/audit.py
10	eval-runner	LLM品質評価・テスト実行	日次 深夜	llm/, tests/
11	orchestrator	他エージェントの調整・スケジューリング	常時	全エージェント

各エージェントの詳細

```
■ Agent 1: knowledge-collector
  入力: 音声ファイル(.wav/.m4a) / テキスト / ドキュメント(.pdf/.xlsx)
  処理: Whisper文字起こし → LLM構造化 → ナレッジ項目生成
  出力: knowledge_items レコード (+ embedding)
  依存: llm/client.py, brain/extraction/
  CLIコマンド: shachotwo knowledge add

■ Agent 2: qa-engine
  入力: 自然言語の質問
  処理: クエリembedding → ベクトル検索 → LLMによる回答生成
  出力: 回答テキスト + 参照ナレッジ + confidence score
  依存: brain/knowledge/, llm/client.py, pgvector
  CLIコマンド: shachotwo qa ask

■ Agent 3: process-miner
  入力: SaaS操作ログ (freee, kintone, Slack等)
  処理: 操作パターン抽出 → 業務フロー推定 → 繰り返しパターン検出
  出力: department_flows レコード + ボトルネック候補
  依存: workers/connector/(ログ取得), brain/inference/
  CLIコマンド: shachotwo bpo discover

■ Agent 4: bpo-executor
  入力: 承認済みワークフロー定義 + トリガーイベント
  処理: Safety Check → (承認待ち) → SaaS API実行 → 結果検証
  出力: BPO実行ログ + 成功/失敗レポート
  依存: workers/bpo/, workers/connector/, process-miner(フロー定義)
  CLIコマンド: shachotwo bpo execute

■ Agent 5: report-generator
  入力: 会社状態スナップショット + 直近のナレッジ変更 + BPO実行結果
  処理: データ集約 → LLMによるサマリー生成 → フォーマット整形
  出力: 日次レポート (LINE通知) / 週次レポート (PDF + LINE)
  依存: brain/knowledge/, brain/twin/
  CLIコマンド: shachotwo report generate

■ Agent 6: risk-detector
  入力: 会社状態 + ナレッジ + 外部情報 (法改正等)
  処理: リスクスコアリング → 閾値超過検出 → 提案生成
  出力: proactive_proposals レコード (risk_alert タイプ)
  依存: brain/proactive/, brain/twin/
  CLIコマンド: shachotwo risk scan

■ Agent 7: genome-builder
  入力: 蓄積ナレッジ + 業種テンプレートのベースライン
  処理: ナレッジ → ゲノム次元マッピング → 充足度計算 → テンプレート更新
  出力: genome_templates レコード更新 + 充足度スコア
  依存: brain/genome/, brain/knowledge/
  CLIコマンド: shachotwo genome build

■ Agent 8: saas-connector
  入力: tool_connections 設定
  処理: OAuth トークンリフレッシュ → ヘルスチェック → ログ取得
  出力: 接続ステータス + 取得ログ
  依存: workers/connector/
  CLIコマンド: shachotwo connector status

■ Agent 9: audit-logger
  入力: 全エージェントのイベントストリーム
  処理: イベント正規化 → 構造化ログ生成 → 保存
  出力: audit_logs レコード
  依存: security/audit.py
  CLIコマンド: shachotwo audit log

■ Agent 10: eval-runner
  入力: ゴールデンデータセット (Q&A: 50件, extraction: 30件)
  処理: テストスイート実行 → 精度計測 → 回帰検出
  出力: 評価レポート (precision/recall/F1 + 前回比較)
  依存: llm/client.py, tests/
  CLIコマンド: shachotwo eval run

■ Agent 11: orchestrator
  入力: スケジュール定義 + イベントトリガー
  処理: エージェント起動 → 依存関係解決 → エラーハンドリング → リトライ
  出力: 実行ログ + ステータスダッシュボード
```

```
  依存: 全エージェント
  CLIコマンド: shachotwo orchestrate run-all
```

エージェント依存関係図

```
orchestrator (Agent 11)
  ├── knowledge-collector (1) ──→ qa-engine (2)
  ├── saas-connector (8) ──→ process-miner (3) ──→ bpo-executor (4)
  ├── genome-builder (7) ←── knowledge-collector (1)
  ├── risk-detector (6) ←── knowledge-collector (1) + process-miner (3)
  ├── report-generator (5) ←── all data sources
  ├── eval-runner (10) ── 独立（日次バッチ）
  └── audit-logger (9) ←── 全エージェント（イベント駆動）
```

---

11. CLIコマンドカタログ

> Phase Aでは全操作をCLIで実行する（ADR-005準拠）。
> GUIは Phase B 以降。CLIログがそのまま監査証跡になる。

ナレッジ操作

```
# 音声ファイルからナレッジを追加（Whisper → LLM構造化）
shachotwo knowledge add --source voice --file recording.wav

# テキストから直接追加
shachotwo knowledge add --source text --content "見積もり100万以上は社長承認"

# ドキュメントから追加（PDF/Excel/Word）
shachotwo knowledge add --source document --file 業務マニュアル.pdf

# ナレッジ検索
shachotwo knowledge search "見積もりの承認フロー"

# カテゴリ別一覧
shachotwo knowledge list --category decision_rules
shachotwo knowledge list --category business_flows --department 営業部

# ナレッジの信頼度確認（Half-life Model）
shachotwo knowledge health --stale-threshold 0.5

# ナレッジ更新（リフレッシュ）
shachotwo knowledge refresh --id <knowledge_id> --content "更新内容"
```

Q&A

```
# 質問
shachotwo qa ask "今月の受注判断基準は？"

# 部門スコープ指定
shachotwo qa ask "残業申請のフローは？" --department 総務部

# バッチ質問（ファイルから）
shachotwo qa batch --file questions.txt --output answers.json
```

BPO（業務自動化）

```
# 業務フロー自動発見（SaaS操作ログ分析）
shachotwo bpo discover --saas freee --days 30

# Shadow Mode起動（バックグラウンドで並行実行）
shachotwo bpo shadow --workflow invoice_check --duration 7d

# Shadow Mode結果確認
shachotwo bpo shadow-report --workflow invoice_check

# ワークフロー実行（ドライラン）
shachotwo bpo execute --workflow invoice_check --dry-run

# ワークフロー実行（本番）
shachotwo bpo execute --workflow invoice_check

# ワークフロー一覧
shachotwo bpo list --status active
```

ゲノム（業種テンプレート）

```
# ゲノム初期化（業種テンプレートから）
shachotwo genome init --industry construction --company-size 30

# ゲノム充足度確認
shachotwo genome status

# ゲノム手動更新
shachotwo genome update --dimension strategy --field vision --value "地域No.1の建設会社"

# ゲノムエクスポート
shachotwo genome export --format json --output my_genome.json

# 他社ゲノムとの比較（匿名化ベンチマーク）
shachotwo genome benchmark --industry construction
```

レポート・提案

```
# 日次レポート生成
shachotwo report generate --type daily

# 週次レポート生成（PDF出力）
shachotwo report generate --type weekly --output weekly_report.pdf

# リスクスキャン（即座に実行）
shachotwo risk scan

# 能動提案一覧
shachotwo proposals list --status pending
shachotwo proposals accept --id <proposal_id>
shachotwo proposals reject --id <proposal_id> --reason "今は優先度低い"
```

SaaS接続管理

```
# SaaS接続追加（OAuth）
shachotwo connector add --saas freee
shachotwo connector add --saas kintone --subdomain mycompany

# 接続状態確認
shachotwo connector status

# ヘルスチェック実行
shachotwo connector healthcheck --all

# 接続削除
shachotwo connector remove --saas slack
```

監査・評価

```
# 監査ログ表示
shachotwo audit log --last 7d
shachotwo audit log --agent bpo-executor --last 24h

# LLM品質評価実行
shachotwo eval run --suite qa_accuracy
shachotwo eval run --suite extraction_quality
shachotwo eval run --suite all

# 評価レポート表示
shachotwo eval report --last-run
```

オーケストレーション

```
# 全エージェント一括実行
shachotwo orchestrate run-all

# 特定エージェントのみ実行
shachotwo orchestrate run --agents "knowledge-collector,qa-engine"

# スケジュール確認
shachotwo orchestrate schedule

# エージェント状態確認
shachotwo orchestrate status
```

関連ドキュメント

ドキュメント	関連内容
c_01_事業計画.md	料金・収支・GTM・競合
c_02_プロダクト設計.md	genome_templates・knowledge_items 等のDB設計
c_03_実装計画.md	Phase別タスクリスト
c_04_営業資料.md	営業トーク・デモシナリオ
b_01_ブレイン編.md	Brain モジュール設計
b_02_BPO編.md	動的エージェントモデル（Phase B以降の正の設計）
c_06_初期顧客共創.md	Phase A' 初期顧客3社との共創詳細
adr/	設計判断記録